Systems Programming: C, Processes & Concurrency

01 / Memory and C → 02 / Processes and IPC → 03 / Threads and safety → 04 / Parallel performance

01 / Representation is a contract

unsigned int and wider wrap modulo max+1, but narrower unsigned types promote to int first; signed overflow is undefined. sizeof returns size_t in C bytes. Initialize automatic objects and match formatted-I/O conversions to actual argument types.

02 / Pointers need live bounds

Array arithmetic stays within the array or its one-past position; never dereference the latter. A multidimensional array decays to a pointer to a row, not a pointer to pointer.

03 / Text needs capacity and termination

Reserve strlen+1 bytes. Bounded reads can split a logical line. Check conversion results, suffixes and range; never treat arbitrary bytes as a terminated C string.

04 / Own every allocation

Check n≤SIZE_MAX/sizeof(*p) before multiplying. Check allocation results, initialize before reading, preserve the old pointer on realloc failure and end all borrowing before one eventual free.

05 / Build stages differ

Preprocessing expands tokens; compilation checks a translation unit; linking resolves external definitions. Headers describe interfaces. Macros can duplicate side effects, and include guards do not prevent cross-file multiple definitions.

06 / Signals need a minimal handler

Prefer sigaction and async-signal-safe operations. A volatile sig_atomic_t stop flag can defer cleanup to normal flow. Use an atomic mask-and-wait protocol when checking a predicate and sleeping for signals.

07 / Process lifecycle

fork duplicates execution with separate ordinary memory; exec replaces the image; waitpid collects status. Check each result, close unused inherited descriptors and decode wait status with its matching macros.

08 / Streams are not messages

Pipes and stream sockets can split or combine writes across reads. Define framing and loop over partial transfers. Pipe EOF requires drained data and every write endpoint closed.

09 / Shared storage needs a protocol

Shared memory removes some copying, not synchronization requirements. Use offsets across mappings. Semaphore permits track resources; mutexes protect invariants. Define initialization, shutdown and recovery ownership.

10 / Thread coordination

Keep arguments alive through use and join created threads correctly. Protect every conflicting access. A condition wait belongs in a predicate loop with its mutex; volatile is not general synchronization.

11 / Prevent circular waits

Acquire locks by stable global rank and include joins, condition waits and I/O in dependency analysis. A trylock loop can avoid blocking yet still livelock; timeouts alone do not diagnose deadlock.

12 / Measure equivalent work

Speedup=T_1/Tp using elapsed time for the same verified result. Partition output ownership, combine after dependencies complete and account for serial work, bandwidth, imbalance and synchronization costs.